

Pacaldo, Alexander John G.

BSCS 3-201

Part 1

1. Search for two (2) free online assemblers available on the Internet. For each assembler, answer the following items:

a. What organization/company manages the online assembler?

1. Jdoodle
2. onlinegdb

b. Provide the links to the online assembler.

<https://www.jdoodle.com/compile-assembler-nasm-online>

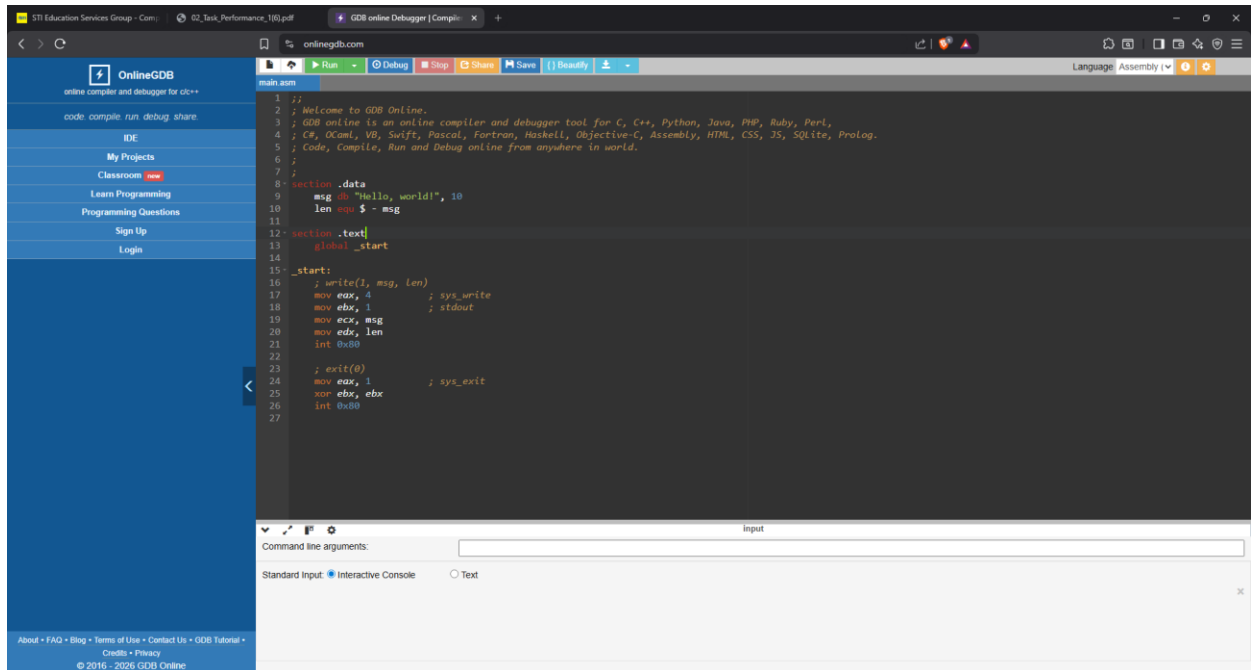
<https://www.onlinegdb.com>

c. Take a screenshot of the initial content of the code editor of the assembler.

JDoodle

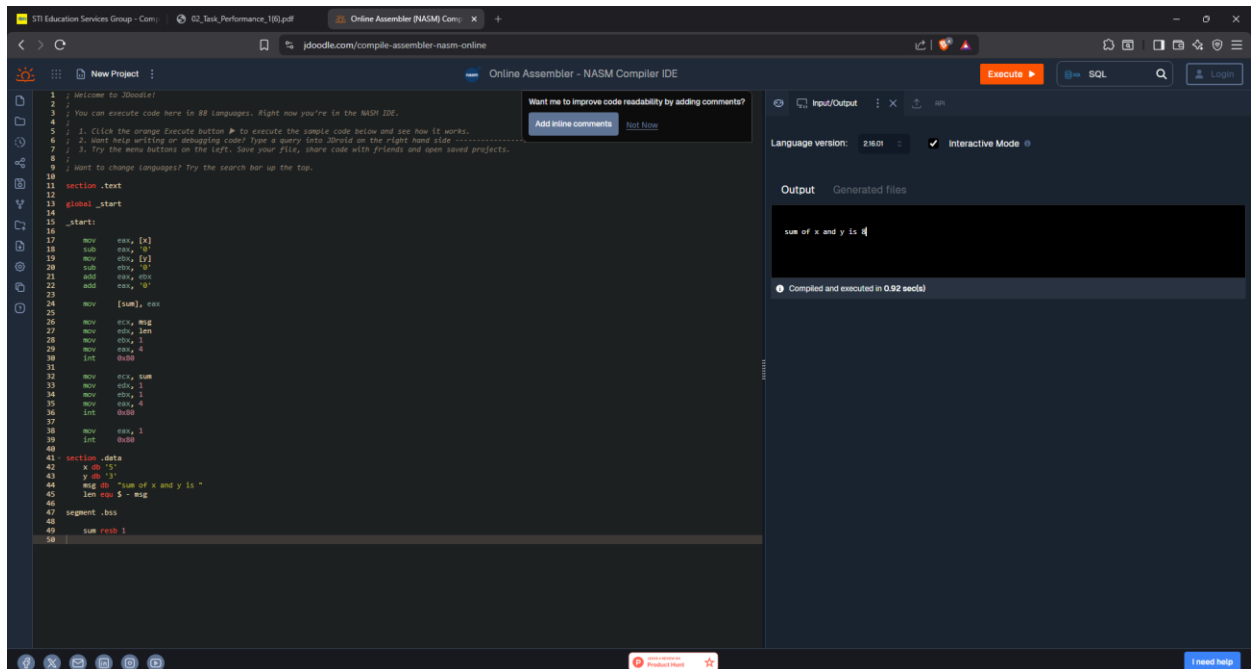
```
1 | welcome to JDoodle!
2 |
3 | You can execute code here in 88 languages. Right now you're in the NASM IDE.
4 |
5 | 1. Click the orange Execute button to execute the sample code below and see how it works.
6 | 2. Want help writing or debugging code? Type a query into 30word on the right hand side.
7 | 3. Try the menu buttons on the left. Save your files, share code with friends and even saved projects.
8 |
9 | Want to change languages? Try the search bar up the top.
10 |
11 |
12 | section .text
13 | global _start
14 |
15 | _start:
16 |
17 | mov     eax, [x]
18 | sub     eax, y
19 | mov     ebx, [y]
20 | sub     ebx, x
21 | add     eax, ebx
22 | add     eax, w
23 |
24 | mov     [sum], eax
25 |
26 | mov     ecx, msg
27 | mov     ebx, len
28 | mov     ebx, 1
29 | mov     edx, 4
30 | int     0x80
31 |
32 | mov     ecx, sum
33 | mov     ebx, 1
34 | mov     ebx, 1
35 | mov     edx, 4
36 | int     0x80
37 |
38 | mov     ebx, 1
39 | int     0x80
40 |
41 | section .data
42 | x db '5'
43 | y db '3'
44 | msg db "sum of x and y is "
45 | len equ $ - msg
46 |
47 | segment .bss
48 |
49 | sum resb 1
50 |
```

OnlineGDB

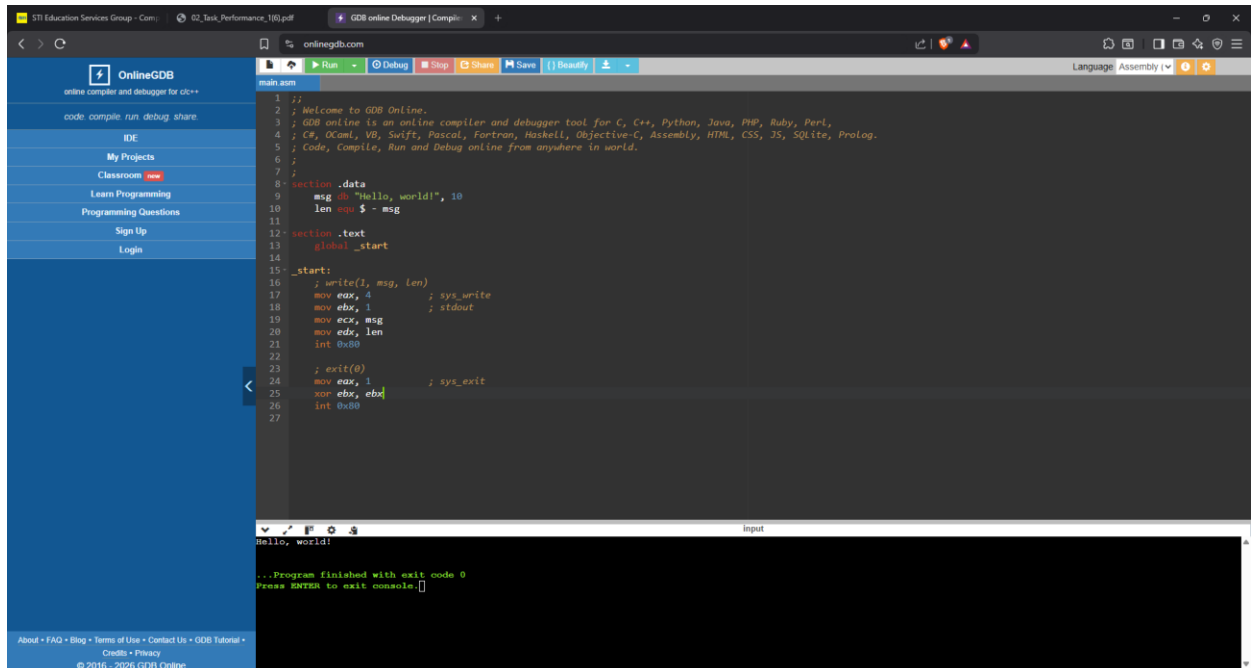


d. Run the initial codes in the editor and take a screenshot of the output/result.

JDoodle



OnlineGDB



e. Does the assembler support an x86 architecture? Elaborate on your answer.

1.Jdoodle

Yes, Jdoodle does support an x86 architecture because it uses NASM and the program uses x86 registers like AX, BX, ECX, EAX, it also supports 32 bit Linux system calls (int 0x80).

2.OnlineGDB

Yes, OnlineGDB does support an x86 architecture because it uses x86 registers like EAX, EBX, ECX. It also supports Linux system calls (int 0x80) and it also follows an x86 (32-bit) architecture.

Part 2

3. Answer the following question regarding the initial assembly codes in the editor:

a. What does the program do?

The program prints a message and shows the result of a calculation. It converts ASCII numbers to real numbers, performs the operation, and converts the result back to ASCII before displaying it.

b. Is the result of the program correct?

Yes, the result of the program is correct because the instructions properly convert and calculate the values.

c. How much memory was utilized in the program execution?

The program used memory in the **.bss section**, where storage was reserved using `resb`. In the program, 2 bytes of memory were reserved for the result.

d. How long did it take to execute the program?

The program executed very fast, almost instantly, because assembly programs are small and run directly on the processor.

e. Under what section were the x and y initialized?

The values were initialized in the `.text` section, where the registers were given values using the `mov` instruction.

5. Execute the program. Then, answer the following items:

a. What does the program do?

The program divides numbers, stores the result, and prints the message "**The result is:**" together with the computed value.

b. How many sections were used in the program?

The program uses three sections:

.text for instructions

.data for initialized data

.bss for reserved memory

c. Identify at least three (3) mnemonics (opcode/directive/macro) that were used in the program.

Three mnemonics used in the program are:

mov

sub

div

d. Identify at least three (3) operands that were used in the program.

Three operands used in the program are:

ax

bl

ecx

e. In the program above, which lines of codes indicate the storage space allocation for the message and message length?

The storage allocation for the message and message length is shown in:

message db "The result is:"

length equ \$ - message

The storage allocation for the result is shown in:

res resb 2